Introduction to Data Science [Section: C]

Midterm Project - Spring 25-26

Project Description
Apply the appropriate data preparation steps to your chosen dataset and compute the relevant descriptive statistics. For this project, each group must work with its own selected dataset and modify it according to the project requirements. You may choose any labelled dataset containing both categorical and numerical attributes (classes will be categorical) that fits the task except iris, titanic, and US Arrest datasets. The project must be completed in a four-person group. A spreadsheet is created in MS Teams to provide all group and project information. The link to the file is given in the Teams group. While giving entry to the spreadsheet. ensure that no other group has already selected your dataset for the project.
Instructions
• The submission deadline for the project is April 5, 2026, 11:59 pm.
• At the beginning of the report, write a short note about the dataset. For each implemented code segment in the R program, provide the code and its output along with their description in the report. In the description part, only write the content (do not write unnecessary content) that is sufficient to understand the code and its output.
• A file related to the report format is provided in MS Teams. Use that file to write the report.
• Comments are not allowed in the R program.
• I will announce the project VIVA schedule in MS Teams. Please bring your implemented program during the VIVA session.
• An assignment is created in MS Teams named "Project - Midterm." Make a ZIP file using your project-related files (R file, Project Report in PDF, and Sample Dataset file) and submit it under that assignment by the deadline. One student from each group will submit the project.
• The following topics can be focused on thinking about the project. Note that the project is not limited to these topics which are mentioned to get an idea about how to proceed with the project.
◦ If there are any missing values in the dataset, we should apply all applicable methods from the available options to handle the missing values.
◦ Detect outliers in the data set and use the appropriate approach to handle those values.
◦ We can convert attributes from numeric to categorical or categorical to numeric.
◦ We can apply the normalization method for any continuous attribute.
◦ We can find and remove duplicate rows.
◦ We can apply some filtering methods to filter the data.
◦ Detect invalid data in the data set and use the appropriate approach to handle those values.
◦ We can convert the imbalanced data set into a balanced data set.
◦ Split the dataset for Training and Testing.
◦ Calculate descriptive statistics and interpret the results for the numerical variables according

to the target classes of your chosen dataset.
◦ Compare the average values of a selected numerical variable across two distinct categories of an appropriate categorical variable from your dataset.
◦ Examine and compare the variability (e.g., IQR,standard deviation, variance, and range) of another numerical variable across the different categories or levels of a chosen categorical variable within your dataset.

Give me a brief of what the question and the file says. and then tell me what should be done. just so you know i need to do this is R language

You have to take a labeled dataset and, using R, do proper data cleaning/preparation and descriptive statistics, then document everything in the given report template and submit it (with R code and dataset) as a zipped project.[1]

Brief of the question

- You must pick a labeled dataset that has both numerical and categorical attributes; the target (class) must be categorical. Iris, Titanic, and US Arrest datasets are not allowed. You also need to ensure no other group has chosen the same dataset (check and fill in the shared spreadsheet on MS Teams).
- The core task is to apply suitable data preparation techniques (handling missing values, outliers, invalid values, duplicates, normalization, encoding, class balancing, filtering, train–test split) and then compute and interpret descriptive statistics for numerical variables, especially *by* target classes.[1]
- You must implement everything in **R**, without comments in the R script, and include both the code and its outputs (with short explanations) inside the report.[1]
- Finally, you create a ZIP file containing:
  - The R script file
  - The project report in **PDF**, using the provided template
  - A sample of your dataset file
    and submit it to the "Project - Midterm" assignment on MS Teams before April 5, 2026, 11:59 pm.[1]

Brief of what the report template says

The attached DOCX file is the official midterm project report template for **Introduction to Data Science**.[1]

- First page: AIUB header and course information (semester, section, group, etc.).[1]
- Group contribution table: you list each member's name, ID, and the parts of the project they contributed to.[1]
- Dataset Description section: a short description of your dataset focusing on features (inputs) and the target variable (class labels).[1]
- Project Implementation Detail section: repeated blocks for each "task" you perform, with the structure:[1]

    - Title of the task
    - Description of the task (what you are solving and how)
    - Code for the task (R code)
    - Sample output of the code
    - Code description (explain key parts of the code and discuss the output briefly)
- At the end: Project Code section where you paste the **entire R code** for the project.[1]

What you should do (step-by-step, in R)

Step 1: Choose and register your dataset

- Pick a supervised learning dataset with:
    - A categorical target (class)
    - At least a few numerical and categorical input features.
- Make sure it's not iris, Titanic, or US Arrest and that no other group has taken it (check/edit Teams spreadsheet).

In R you will eventually load it using something like: `read.csv("your_dataset.csv")` or similar.

Step 2: Understand and describe the dataset

- In R:
    - Inspect structure: `str(df)`, `head(df)`.
    - Check summary: `summary(df)`.
- In the report's "Dataset Description" section, explain:
    - What each key attribute means
    - Which column is the target class
    - Basic info: number of rows, number of features, type of problem (classification).

Step 3: Data cleaning and preparation tasks

For each of the following tasks, you will:

1. Decide what exactly you're doing on your dataset.
2. Implement it in R.
3. Capture code, output, and a short explanation in one "task block" in the template.

Typical tasks to include:

1. **Handle missing values**
    - Detect: `colSums(is.na(df))`.
    - Apply several applicable methods (e.g., removing rows, imputing with mean/median/mode, or using more advanced techniques).

- Show before/after summary.

2. **Detect and treat outliers**
   - Use boxplots: `boxplot(df$numeric_var)` or IQR-based rules.
   - Decide to cap, remove, or transform outliers.
3. **Convert variable types (encoding)**
   - Numeric → categorical (e.g., binning ages into groups with `cut`).
   - Categorical → numeric (e.g., using `factor()` or dummy variables with `model.matrix` or `fastDummies`).
4. **Normalization / scaling of continuous attributes**
   - Apply something like `scale()` to selected continuous variables.
5. **Remove duplicates**
   - Detect: `duplicated(df)`; remove: `df <- df[!duplicated(df), ]`.
6. **Filter the data**
   - Apply some meaningful filters, e.g., keep only realistic ranges or specific categories.
7. **Detect invalid data and fix it**
   - Check for impossible values (e.g., negative ages, invalid category labels).
   - Correct or remove such records.
8. **Balance the target classes (if imbalanced)**
   - Check class distribution: `table(df$target)`.
   - Apply oversampling/undersampling (e.g., using `ROSE`, `DMwR`, `caret` or simple custom up/down-sampling).
9. **Split into training and testing sets**
   - Use a function like `set.seed(…)` and `sample()` or `caret::createDataPartition()` to create train/test splits (e.g., 70/30).

Each of these can be one or more tasks in the "Project Implementation Detail" section.

Step 4: Descriptive statistics and comparisons

You must do at least these three analyses in R:

1. **Descriptive statistics by target classes**
   - For each numerical variable, compute statistics separately for each class label:
     - Mean, median, standard deviation, min, max, etc.
   - In R, you can use `aggregate()`, `tapply()`, or `dplyr::group_by(target) %>% summarise(...)`.
   - Interpret what the differences tell you about the classes.
2. **Compare means across two categories**
   - Choose one numerical variable and one categorical variable with two categories.

    - Compare the average of that numerical variable between those two categories (e.g., with `aggregate()` or `dplyr`).
    - Optionally, do a simple t-test (`t.test()`) if appropriate and interpret.
3. **Compare variability across categories**
    - Pick another numerical variable and a categorical variable with multiple levels.
    - Compute IQR, standard deviation, variance, and range of that numeric variable for each category level.
    - In R:
        - IQR: `IQR(x)`
        - Standard deviation: `sd(x)`
        - Variance: `var(x)`
        - Range: `max(x) - min(x)`
    - Summarise these per category and comment on which category is more/less variable and what that might imply.

For each of these, create a separate task in the report with: title, task description, R code, output, and explanation.

Step 5: Organize the report using the template

- Fill in the cover info (course, section C, group, semester) and group contribution table.
- For every chosen task (missing values, outliers, etc., plus descriptive stats tasks):
    - Write "Title of the task" (e.g., "Handling Missing Values in Attribute X").
    - Describe what/why you're doing it in 2–4 concise sentences.
    - Paste the R code for that task only.
    - Paste relevant output (console output, summaries, maybe small tables).
    - Explain key parts of the code and interpret the output briefly.
- At the end, paste the full R script under "Project Code".

Step 6: Final packaging and submission

- Export the report as **PDF** from Word.
- Ensure your R script has **no comments**.
- Prepare a sample of your dataset (e.g., CSV).
- Create a ZIP with: `your_project.R`, `your_report.pdf`, `sample_dataset.csv`.
- Submit the ZIP to the "Project - Midterm" assignment in MS Teams before the deadline, and be ready to show and explain your R code in the VIVA.

If you want, next we can:

- Pick a concrete dataset together and design an exact list of tasks.

- Draft example R code (without comments) for each required step that you can adapt.

⁂

So you are the one who will tell me step by steps and code and i will do such starting from how to open r studio to the very last end of the project. See the names here and at first step you told me to find dataset so choose dataset for me. Thank you. Start

Nice, we'll walk through everything step by step and I'll give you exact R code you can copy and run.

First, let's fix the dataset choice.

Dataset we will use

I recommend the **UCI Bank Marketing Dataset** (classification: predict if a bank client subscribes a term deposit).
It has:

- Categorical features (job, marital status, education, etc.) and numerical features (age, duration, number of contacts, etc.).[2]
- Binary categorical target variable `y` with values `"yes"` or `"no"` (whether the client subscribed the deposit).[3]
- Publicly available versions on UCI and Kaggle (e.g., "UCI Bank Marketing Dataset" on Kaggle).[4] [5]

This matches your project requirements (labelled data, categorical classes, mix of numeric and categorical, not in the banned list).

From your class spreadsheet, no group has taken **Bank Marketing** yet (they took heart disease, adult income, loan approval, churn, stroke, etc.).[6]

Immediate actions for you (before coding)

1. **Confirm with your group that you'll use this dataset.**
2. **Register it in the class spreadsheet** (same Excel file you showed, but in MS Teams):
   - Dataset Name: `UCI Bank Marketing Dataset`
   - Dataset Link: for example, paste a Kaggle link like:
     - `https://www.kaggle.com/datasets/adityamhaske/bank-marketing-dataset` or another UCI bank marketing link.[5] [4]

3. Download the main CSV (often called `bank-additional-full.csv` or similar) from Kaggle/UCI into a new folder on your PC, e.g.:
   `C:\IDS_Midterm_BankMarketing\`

Once this is done, we start RStudio.

Step 1: Open RStudio and set up project

Do this once:

1. **Open RStudio**
   - Click the **RStudio** icon from Start Menu or Desktop.
2. **Create a new project** (optional but recommended):
   - Top menu: `File` → `New Project…`
   - Choose **Existing Directory**.
   - Browse to your project folder (e.g. `C:\IDS_Midterm_BankMarketing\`) where you put the CSV.
   - Click **Create Project**.
3. Make sure your dataset file (for example `bank-additional-full.csv`) is inside this folder.

Now, in the bottom-left **Console** of RStudio you can run R code.

Step 2: First R code – load the dataset

In RStudio, go to **File** → **New File** → **R Script** and paste this code (adjust the file name if needed):

```
# Set working directory to your project folder if needed
setwd("C:/IDS_Midterm_BankMarketing")

# Read the CSV (change the name if your file is different)
bank <- read.csv("bank-additional-full.csv", sep = ";", header = TRUE)

# Quick checks
dim(bank)          # number of rows and columns
names(bank)        # column names
str(bank)          # structure: types of variables
summary(bank)      # basic descriptive statistics
```

Then click **Run** (or select all and press `Ctrl+Enter`).

This will:

- Load the bank marketing data into an object `bank`.
- Show you how many rows/columns you have and each variable's type (numeric, factor, character, etc.).

Step 3: How we will proceed (overall roadmap)

I'll guide you in this order in next messages, each with ready-to-use R code:

1. **Basic exploration and dataset description**
   - Understanding columns, target `y`, and types.
2. **Handling missing values**
3. **Detecting and treating outliers**
4. **Type conversions (numeric ↔ categorical, encoding)**
5. **Normalization of continuous variables**
6. **Removing duplicates and filtering invalid data**
7. **Handling class imbalance (if needed) and train–test split**
8. **Descriptive statistics by target class** `y`
9. **Comparing means across two categories**
10. **Comparing variability (IQR, sd, variance, range) across categories**
11. **Organizing everything into your Word report using the given template.**

What I need from you now

1. Confirm:
   - "Yes, we'll use UCI Bank Marketing Dataset and I have downloaded the CSV into a folder."
2. Tell me the **exact CSV file name** you downloaded (e.g., `bank-additional-full.csv` or something else).

After that, I'll give you the next block of **exact R code** for:

- Clean loading of the data
- Converting key variables to proper types
- Writing the **Dataset Description** part for your report.

⁂

```
# <q>Step 1: Open RStudio and set up projectDo this once:Open RStudioClick the RStudio
```

You're not in RStudio yet—you've opened **base R (RGui)**. That's why you don't see "New Project...". We have two options; I'll show both and you can choose the easier for you.[18] [19]

Option A (recommended): Install and use RStudio

RStudio is a separate program (IDE) that sits on top of R and gives you the interface I was describing.[19] [18]

A1. Check if RStudio is installed

On Windows:

- Press the **Windows key**, type `RStudio`.
- If you see an app called **RStudio** with a blue icon, click that.
- When it opens, the window should say "RStudio" in the title bar and you should see multiple panes (Console, Environment, etc.).[18]

If you find it, just tell me "I'm in RStudio now", and we'll continue (I'll adjust steps without projects if you want).

A2. If RStudio is NOT installed

Then you only installed R 4.5.3, not RStudio (this is normal).[20] [21]

Do this:

1. Open your browser and go to: `https://posit.co/download/rstudio-desktop/` (RStudio Desktop download page).
2. Download the **RStudio Desktop for Windows** installer.
3. Run the installer (Next → Next → Finish, default options are fine).
4. After installation, open **RStudio** from the Start Menu.

Once RStudio is open, you'll see:

- Top left: empty or script area.
- Bottom left: Console.
- Top right: Environment/History.
- Bottom right: Files/Plots/Packages/Help.[18]

Reply when you reach this point, and tell me:

- "RStudio is open."
- And where you put the CSV (e.g., `C:\IDS_Midterm_BankMarketing\bank-additional-full.csv`).

Then I'll:

- Skip the complex "Project" step for now (not mandatory).
- Give you **simple, exact commands** to set working directory and load the dataset directly in RStudio.

Option B: Continue in base R (if you really want)

If you don't want to install RStudio right now, we can still do the project in the RGui you have open.[20]

In that case:

1. Save your CSV somewhere, e.g. `C:\IDS_Midterm_BankMarketing\bank-additional-full.csv`.
2. In the R console window you see now, run (change the path to yours):

```
setwd("C:/IDS_Midterm_BankMarketing")
bank <- read.csv("bank-additional-full.csv", sep = ";", header = TRUE)
dim(bank)
```

But RStudio will make your life much easier for a semester-long course, so I strongly recommend Option A.

Next: Tell me exactly what you see

Please answer with:

- Whether you have installed RStudio or only R.
- What you see in the window title (e.g., "R x64 4.5.3" vs "RStudio").

Then I'll adjust the next steps and code to match your exact screen and we'll continue smoothly.

⁂

i have saved my csv somewhere, i can find it you tell me what to do, i have opened r studio

You're currently in **RGui (base R)**, not RStudio—that's why the interface doesn't match what I described earlier. That's totally fine; we can do the whole project from here.[33] [34]

Step 1: Load your CSV using a file dialog

Since you're not sure about the folder path, we'll let R open a file browser:

1. Click inside the **R console** (the white window with `>` prompt).
2. Type these two lines one by one (press Enter after each):

```
data_path <- choose.files()
bank <- read.csv(data_path, sep = ";", header = TRUE)
```

- After `choose.files()` runs, Windows will open a dialog.

- Navigate to the folder where you saved your CSV, click the file (e.g., `bank-additional-full.csv`), and press **Open**.
- R will automatically store the full path in `data_path` and then read the CSV into a data frame called `bank`.[33]

If your CSV uses comma instead of semicolon, change `sep = ";"` to `sep = ","`.

Step 2: Verify that the data loaded

Right after the previous code, run:

```
dim(bank)       # rows and columns
names(bank)     # column names
str(bank)       # types of variables
head(bank)      # first few rows
```

You should see:

- A dimension like `41188 21` (example) for rows × columns.
- A list of column names including things like `age`, `job`, `marital`, `education`, and target `y` (if you're using the bank marketing dataset).[35] [36]

Tell me:

- Whether `bank` loaded without error.
- What `dim(bank)` prints.
- The **exact name of your CSV file** (as shown in the dialog).

Once this is working, the next step will be:

- Cleaning types (making sure `y` and other categoricals are factors).
- Then we'll start with the first project task ("Dataset Description") using this loaded object.

⁂

no no tell me how to open rstudio

Perfect, let's get you into **RStudio** (the IDE) instead of the RGui you're seeing now.

Step 1 – Check if RStudio is already installed

1. Press the **Windows key** on your keyboard.
2. Type `RStudio`.
3. If you see an app called **RStudio** (blue icon) in the list:
    - Click it.

- A new window should open with "RStudio" in the title bar and multiple panes (Console, Environment, Files, etc.).[38]

If that works, you're done with installation — just reply:

> "RStudio opened"

and I'll give you the next steps (loading your CSV, etc.) for that screen.

If you **do not** see RStudio in the Start menu, continue with Step 2.

Step 2 – Download RStudio Desktop for Windows

Since you already have R 4.5.3, you only need to install RStudio.[39]

1. Open your browser (Chrome/Edge).
2. Go to this official link: `https://posit.co/download/rstudio-desktop/`[40] [39]
3. On that page, find "**RStudio Desktop**" (usually section "2: Install RStudio") and click the **Download** button for **Windows**.[39] [40]
4. A file like `RStudio-2026.01.2-418.exe` will download (name can vary).[40]

Step 3 – Install RStudio

1. When the download finishes, click the downloaded `.exe` file (from the browser bottom bar or your Downloads folder).
2. Windows will ask "Allow this app to make changes?" → click **Yes**.[41]
3. RStudio Setup Wizard appears:
   - Click **Next**.
   - Keep the default installation folder → **Next**.
   - Keep default Start Menu folder → **Install**.[41]
4. Wait for it to finish, then click **Finish**. RStudio is now installed.[41]

Step 4 – Open RStudio

1. Press **Windows key**, type `RStudio` again.
2. Click the **RStudio** app.
3. On first run, it may ask which R to use; just keep the default (your installed R 4.5.3) and click **OK**.[41]

You should now see:

- Top-left: Source/Script area (maybe empty).
- Bottom-left: **Console** tab with an > prompt.

- Top-right: Environment/History.
- Bottom-right: Files/Plots/Packages/Help.[38]

What to send me next

Once you see this RStudio interface, reply:

- "RStudio is open now."

Then I'll immediately give you:

- The exact commands to load your CSV (without needing to know the path).
- And we'll start building your project in RStudio step by step.

⁂

yes and i also found the new project options, now what to do

Great, now we're really in business.

We'll do four things now:

1. Make a project folder.
2. Create an RStudio Project in that folder.
3. Create and save an R script.
4. Load your CSV in that script.

Step 1 – Create a folder for your project (Windows)

Outside RStudio:

1. Open **File Explorer**.
2. Create a folder, for example:
   `Documents → IDS_Midterm_BankMarketing`
3. Move your dataset CSV file into this folder (drag-and-drop).

Remember the exact file name (e.g. `bank-additional-full.csv` or something else) — we'll need it.

Step 2 – Create an RStudio Project in that folder

In **RStudio**:

1. Top menu: **File → New Project...**.
2. Choose **Existing Directory**.[54] [55]

3. Click **Browse…**, navigate to the folder you just created (`IDS_Midterm_BankMarketing`), select it, and click **Open**.
4. Click **Create Project**.

RStudio will restart into that project; at the top right you should now see `Project: IDS_Midterm_BankMarketing` (or whatever folder name you chose).[54]

Bottom-right **Files** pane should show your CSV file.

Step 3 – Create and save your main R script

1. Top menu: **File** → **New File** → **R Script**.
2. A blank script opens in the top-left editor.
3. Save it: **File** → **Save** (or Ctrl+S).
4. Name it something like `midterm_project.R` (it will be saved inside your project folder).

This script will contain *all* the code for your project.

Step 4 – First code: load the dataset

In that script, type this code (we will remove comments later if needed):

```
# Replace the file name with your actual CSV name
bank <- read.csv("YOUR_FILE_NAME.csv", sep = ";", header = TRUE)

dim(bank)
names(bank)
str(bank)
summary(bank)
```

- Replace `YOUR_FILE_NAME.csv` with your actual file name exactly (including `.csv`).
- If your file uses comma instead of semicolon, change `sep = ";"` to `sep = ","`.

Then:

1. Select all these lines (Ctrl+A in the script).
2. Press **Ctrl+Enter** to run them, or click **Run**.

You should see output in the **Console** (bottom-left pane).[56]

What I need from you next

1. Confirm that:
   - The project is created and you see your CSV in the Files pane.
   - You created and saved `midterm_project.R`.
   - You ran the code and it worked.

2. Tell me:
    - The **exact CSV file name** you used in the code.
    - The result of `dim(bank)` (just copy the line from the Console).

Then I'll:

- Fix the `sep` if needed,
- And move to the next step: making sure variables are the right types and writing the **Dataset Description** section for your report.

⁂

E:\developer\data_project\IDS_Midterm_BankMarketing

this is where the csv files are

I have two datas
bank.csv 451KB
bankl-full.csv 4503KB

Nice, that path is perfect. We'll now point RStudio to that folder and load **bankl-full.csv** (the bigger one) as our main dataset.

Step 1 – Make RStudio use that folder

In **RStudio**:

1. Top menu: **Session → Set Working Directory → Choose Directory…**.
2. In the dialog, navigate to:
   `E:\developer\data_project\IDS_Midterm_BankMarketing`
3. Select that folder and click **OK**.

Now your working directory in R is that path (you'll see a `setwd("E:/developer/data_project/IDS_Midterm_BankMarketing")` command appear in the Console; R uses forward slashes).[57]

(If you want, you can instead create a New Project in that folder using **File → New Project → Existing Directory → Browse to that same path**, but just setting the working directory is enough for now.)[58] [59]

Step 2 – Load the full dataset in your script

Open your script `midterm_project.R` and replace the previous read code with this (or add below it):

```
# Working directory is already set via Session -> Set Working Directory
# Now load the full bank marketing dataset

bank <- read.csv("bankl-full.csv", sep = ";", header = TRUE)

dim(bank)
names(bank)
str(bank)
summary(bank)
```

Notes:

- Use the file name **exactly** as it appears in the Files pane (`bankl-full.csv` – if that "l" is actually a "1" or something else, match it exactly).
- If `summary(bank)` prints weird values (all in one column), we'll adjust `sep`, but for UCI bank marketing it is usually `;`.

Run these lines (Ctrl+A → Ctrl+Enter).

Step 3 – Tell me what you see

From the Console, copy back:

- The output of `dim(bank)` (e.g., `41188 21`).
- Confirm there was **no error** when running `read.csv`.

Once I have that, next step will be:

- Fixing variable types (make targets and categoricals into factors).
- And I'll give you the exact text to paste into your **Dataset Description** section of the report.

⁂

> # Working directory is already set via Session -> Set Working Directory

Now load the full bank marketing dataset

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)
dim(bank)
[1] 45211 17
names(bank)
[1] "age" "job" "marital" "education" "default" "balance" "housing" "loan" "contact"
[10] "day" "month" "duration" "campaign" "pdays" "previous" "poutcome" "y"
str(bank)
'data.frame':45211 obs. of 17 variables:
$ age : int 58 44 33 47 33 35 28 42 58 43 ...
$ job : chr "management" "technician" "entrepreneur" "blue-collar" ...
$ marital : chr "married" "single" "married" "married" ...
$ education: chr "tertiary" "secondary" "secondary" "unknown" ...
$ default : chr "no" "no" "no" "no" ...
$ balance : int 2143 29 2 1506 1 231 447 2 121 593 ...
$ housing : chr "yes" "yes" "yes" "yes" ...
$ loan : chr "no" "no" "yes" "no" ...
$ contact : chr "unknown" "unknown" "unknown" "unknown" ...
$ day : int 5 5 5 5 5 5 5 5 5 5 ...
$ month : chr "may" "may" "may" "may" ...
$ duration : int 261 151 76 92 198 139 217 380 50 55 ...
$ campaign : int 1 1 1 1 1 1 1 1 1 1 ...
$ pdays : int -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 ...
$ previous : int 0 0 0 0 0 0 0 0 0 0 ...
$ poutcome : chr "unknown" "unknown" "unknown" "unknown" ...
$ y : chr "no" "no" "no" "no" ...
summary(bank)
age job marital education default balance
Min. :18.00 Length:45211 Length:45211 Length:45211 Length:45211 Min. : -8019
1st Qu.:33.00 Class :character Class :character Class :character Class :character 1st Qu.: 72
Median :39.00 Mode :character Mode :character Mode :character Mode :character Median
: 448
Mean :40.94 Mean : 1362
3rd Qu.:48.00 3rd Qu.: 1428
Max. :95.00 Max. :102127
housing loan contact day month duration
Length:45211 Length:45211 Length:45211 Min. : 1.00 Length:45211 Min. : 0.0
Class :character Class :character Class :character 1st Qu.: 8.00 Class :character 1st Qu.:
103.0
Mode :character Mode :character Mode :character Median :16.00 Mode :character Median
: 180.0
Mean :15.81 Mean : 258.2
3rd Qu.:21.00 3rd Qu.: 319.0
Max. :31.00 Max. :4918.0
campaign pdays previous poutcome y
```

```
 Min.   : 1.000   Min.   : -1.0   Min.   : 0.0000   Length:45211       Length:45211
 1st Qu.: 1.000   1st Qu.: -1.0   1st Qu.: 0.0000   Class :character   Class :character
 Median : 2.000   Median : -1.0   Median : 0.0000   Mode  :character   Mode  :character
 Mean   : 2.764   Mean   : 40.2   Mean   : 0.5803
 3rd Qu.: 3.000   3rd Qu.: -1.0   3rd Qu.: 0.0000
 Max.   :63.000   Max.   :871.0   Max.   :275.0000
```

Perfect — the data is loaded correctly and this is exactly the classic **Bank Marketing** dataset (45,211 rows, 17 variables, target `y`).[60] [61]

Now we'll do three things:

1. Convert character columns to categorical (factors).
2. Quickly check the target variable.
3. Write the **Dataset Description** text for your report.

Step 1 – Convert character columns to factors

In your `midterm_project.R` script, **below** what you already have, add and run this code:

```
# Convert character variables to factors (categorical)
factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")

bank[factor_cols] <- lapply(bank[factor_cols], factor)

str(bank)
```

- This keeps numeric columns (`age`, `balance`, `duration`, etc.) as numbers.
- It turns `job`, `marital`, `education`, `housing`, `loan`, `contact`, `month`, `poutcome`, and the target `y` into factors (categorical), which is what you want for descriptive stats by class.[61] [60]

Check in the Console that `str(bank)` now shows `Factor w/ ... levels` for those columns, especially `y`.

Step 2 – Inspect the target variable `y`

Still in the same script, add and run:

```
table(bank$y)
prop.table(table(bank$y))
```

You should see counts of `"no"` and `"yes"` and their proportions.
This will be useful later when we talk about imbalance and balancing methods.

Step 3 – Dataset Description text for your report

Based on what you loaded (`45211` rows, 17 variables) and the known description of this dataset on UCI/Kaggle, here is text you can paste into the "**Dataset Description**" section of your report (you can lightly rephrase if you like):

> The dataset used in this project is the Bank Marketing dataset from a Portuguese retail bank's direct marketing campaigns. It contains 45,211 client records and 17 attributes describing demographic information, financial status, and details about the last marketing contact. Numerical attributes include age, account balance, duration of the last call in seconds, number of contacts during the current campaign, number of days since the client was last contacted in a previous campaign, and the number of previous contacts. Categorical attributes capture the client's job type, marital status, education level, credit default status, housing loan status, personal loan status, contact communication type, last contact month, outcome of any previous campaign, and the target variable `y`, which indicates whether the client subscribed to a term deposit (`yes` or `no`).[60] [61]

You can add one more sentence like:

> This dataset is appropriate for supervised classification with a categorical target and a mix of numerical and categorical predictors, which aligns with the project requirements.[61] [60]

What next?

Reply with:

- Whether `str(bank)` now shows `Factor` for `y` and other categorical variables.
- The output of `table(bank$y)`.

Then I'll guide you through your **first project task block**: handling missing values (even if they appear as `"unknown"` instead of `NA`) and how to present that code + output in the report.

⁂

<q>Step 1 – Convert character columns to factorsIn your midterm_project.R script, below what you already have, add and run this code:r# Convert character variables to factors (categorical)

factor_cols <- c("job", "marital", "education", "default",
"housing", "loan", "contact", "month",
"poutcome", "y")

bank[factor_cols] <- lapply(bank[factor_cols], factor)

str(bank)This keeps numeric columns (age, balance, duration, etc.) as numbers.It turns job, marital, education, housing, loan, contact, month, poutcome, and the target y into factors (categorical), which is what you want for descriptive stats by class.Check in the Console that str(bank) now shows Factor w/ … levels for those columns, especially y.Step 2 – Inspect the target variable yStill in the same script, add and run:rtable(bank$y)$prop.table(table(bank$y))$You should see counts of "no" and "yes" and their proportions.This will be useful later when we talk about imbalance and balancing methods.</q> > # Convert character variables to factors (categorical)

```
factor_cols <- c("job", "marital", "education", "default",
```

- ```
  "housing", "loan", "contact", "month",
  ```

- ```
  "poutcome", "y")
  ```

```
bank[factor_cols] <- lapply(bank[factor_cols], factor)
str(bank)
'data.frame':45211 obs. of 17 variables:
$ age : int 58 44 33 47 33 35 28 42 58 43 …
$ job : Factor w/ 12 levels "admin.","blue-collar",..: 5 10 3 2 12 5 5 3 6 10 …
$ marital : Factor w/ 3 levels "divorced","married",..: 2 3 2 2 3 2 3 1 2 3 …
$ education: Factor w/ 4 levels "primary","secondary",..: 3 2 2 4 4 3 3 3 1 2 …
$ default : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 2 1 1 …
$ balance : int 2143 29 2 1506 1 231 447 2 121 593 …
$ housing : Factor w/ 2 levels "no","yes": 2 2 2 2 1 2 2 2 2 2 …
$ loan : Factor w/ 2 levels "no","yes": 1 1 2 1 1 1 2 1 1 1 …
$ contact : Factor w/ 3 levels "cellular","telephone",..: 3 3 3 3 3 3 3 3 3 3 …
$ day : int 5 5 5 5 5 5 5 5 5 5 …
$ month : Factor w/ 12 levels "apr","aug","dec",..: 9 9 9 9 9 9 9 9 9 9 …
$ duration : int 261 151 76 92 198 139 217 380 50 55 …
$ campaign : int 1 1 1 1 1 1 1 1 1 1 …
$ pdays : int -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 …
$ previous : int 0 0 0 0 0 0 0 0 0 0 …
$ poutcome : Factor w/ 4 levels "failure","other",..: 4 4 4 4 4 4 4 4 4 4 …
$ y : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 …
table(bank$y)
```

no yes
39922 5289

```
prop.table(table(bank$y))
```

```
no        yes
```

0.8830152 0.1169848

"The dataset used in this project is the Bank Marketing dataset from a Portuguese retail bank's direct marketing campaigns. It contains 45,211 client records and 17 attributes describing demographic information, financial status, and details about the last marketing contact. Numerical attributes include age, account balance, duration of the last call in seconds, number of contacts during the current campaign, number of days since the client was last contacted in a previous campaign, and the number of previous contacts. Categorical attributes capture the client's job type, marital status, education level, credit default status, housing loan status, personal loan status, contact communication type, last contact month, outcome of any previous campaign, and the target variable y, which indicates whether the client subscribed to a term deposit (yes or no)."

This is exactly what i wrote in the word file

Everything you did is correct so far: the variables are now proper factors and you've confirmed the class imbalance (about 88% "no", 12% "yes").[62] [63]

Your dataset description text is good and accurate. To make it a bit richer (and clearly tied to your actual data), you can add one extra sentence at the end, like:

> In this sample, 39,922 clients did not subscribe and 5,289 did, so only about 11.7% of clients said "yes" to the term deposit, indicating a highly imbalanced target variable `y`. [63] [62]

(You already have the numbers from `table(bank$y)` and `prop.table(table(bank$y))`.)

Next task: Missing & "unknown" values (Task 1 for report)

The original project hints say: handle missing values and detect invalid data.
In this dataset there are usually **no actual NA values**, but many `"unknown"` entries in categorical columns that behave like missing values.[62] [63]

We'll treat this as **Task 1** in your report.

1. Code to check NA and "unknown"

Add this to your script after what you already have and run it:

```
# Check true NA values
na_counts <- colSums(is.na(bank))
na_counts

# Check 'unknown' levels in categorical variables
factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")

unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))
unknown_counts
```

What this does:

- `na_counts`: shows if there are any real `NA` values.
- `unknown_counts`: shows how many `"unknown"` entries each categorical attribute has.

You will paste both outputs into your report under this task.

2. Clean "unknown" in a sensible way

We will create a cleaned dataset `bank_clean` so the original `bank` stays untouched.

For example strategy:

- For **education**: drop rows where education is `"unknown"` (treat as missing and remove).
- For **job**: replace `"unknown"` with the most common job (mode) to show a simple imputation method.

Add and run:

```
bank_clean <- bank

# Remove rows with unknown education
bank_clean <- subset(bank_clean, education != "unknown")

# Impute unknown job with most frequent job
job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[^9_1]
job_mode

bank_clean$job[bank_clean$job == "unknown"] <- job_mode

# Check remaining unknowns
sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))

# New dimension after cleaning
dim(bank_clean)
```

You'll get:

- `job_mode`: the most frequent job category.
- Updated counts of `"unknown"` after cleaning.
- New `dim(bank_clean)` (rows reduced because we dropped some).

How to write this as a task in the report

In the "Project Implementation Detail" section, create a block:

Title of the task
"Handling Missing and Unknown Values in Categorical Attributes"

Description of the task
(2–4 sentences, for example)

> In this task, I investigate missing values and "unknown" entries in the bank marketing dataset. First, I check each attribute for actual NA values. Then, for categorical variables, I count how many times the level `"unknown"` appears, treating it as a form of missing information. I remove records with unknown education and impute unknown job values with the most frequent job category to obtain a cleaner dataset for further analysis.[63] [62]

Code for the task
Paste exactly the R code you used for:

- `na_counts`
- `unknown_counts`
- creation of `bank_clean`, `job_mode`, and the final checks.

(Remember: for the final R file you'll submit, you should remove the `#` comments, but in the report screenshots/snippets it's fine—your instructor mainly cares that the `.R` file has no comments.)

Sample output of your code
Paste the console outputs:

- `na_counts`
- `unknown_counts`
- `job_mode`
- the new `dim(bank_clean)` and the final unknown counts.

Code description of the task
Write 3–5 lines explaining, for example:

- That `colSums(is.na(bank))` showed no actual NA values.
- That some categorical variables had many `"unknown"` values (especially education, job, etc.).
- That you chose to remove records with unknown education and impute unknown job with the mode.
- That after cleaning, the dataset has `XXXX` rows and fewer or no `"unknown"` values.

Your next action

1. Run the new code, then copy here:
    - `na_counts` output.
    - `unknown_counts` output.
    - `dim(bank_clean)`.

2. After that, I'll guide you through **Task 2: Outlier detection and handling** (with ready R code and how to write it in the report).

⁂

"> # Check true NA values

> na_counts <- colSums(is.na(bank))
> na_counts
> age job marital education default balance housing loan contact day month duration
> 0 0 0 0 0 0 0 0 0 0 0 0
> campaign pdays previous poutcome y
> 0 0 0 0 0
>
> # Check 'unknown' levels in categorical variables
>
> factor_cols <- c("job", "marital", "education", "default",

- ```
  "housing", "loan", "contact", "month",
  ```

- ```
  "poutcome", "y")
  ```

```
unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))
unknown_counts
job marital education default housing loan contact month poutcome y
288 0 1857 0 0 0 13020 0 36959 0
bank_clean <- bank
```

Remove rows with unknown education

```
bank_clean <- subset(bank_clean, education != "unknown")
```

Impute unknown job with most frequent job

```
job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[1]
job_mode
[1] "blue-collar"
bank_clean$job[bank_clean$job == "unknown"] <- job_mode
```

Check remaining unknowns

```
sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))
job marital education default housing loan contact month poutcome y
0 0 0 0 0 0 12343 0 35425 0
```

New dimension after cleaning

```
dim(bank_clean)
[1] 43354 17"
```

This is the output i got after adding the new two new files and run them, this is my present r script:

"# Working directory is already set via Session -> Set Working Directory

Now load the full bank marketing dataset

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)

dim(bank)
names(bank)
str(bank)
summary(bank)
```

Convert character variables to factors (categorical)

```
factor_cols <- c("job", "marital", "education", "default",
"housing", "loan", "contact", "month",
"poutcome", "y")

bank[factor_cols] <- lapply(bank[factor_cols], factor)

str(bank)

table(bank$y)prop.table(table(bank$y))
```

Check true NA values

```
na_counts <- colSums(is.na(bank))
na_counts
```

Check 'unknown' levels in categorical variables

```
factor_cols <- c("job", "marital", "education", "default",
"housing", "loan", "contact", "month",
"poutcome", "y")

unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))
unknown_counts

bank_clean <- bank
```

Remove rows with unknown education

```
bank_clean <- subset(bank_clean, education != "unknown")
```

Impute unknown job with most frequent job

```
job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[1]
job_mode

bank_clean$job[bank_clean$job == "unknown"] <- job_mode
```

Check remaining unknowns

```
sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))
```

New dimension after cleaning

dim(bank_clean)"

Tell me what should i write for:

"Title of the task1
Description of the task1: which task you are going to solve here and how you will solve it.

Code for task1

Sample output of your code for task1

Code description of task1: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

"

Here's exactly what you can put in the report for **Task 1** based on your current code and outputs.

Title of the task 1

Handling Missing and "Unknown" Values in the Bank Marketing Dataset

Description of the task 1

> In this task, I investigate missing values in the bank marketing dataset and clean the categorical attributes for further analysis. First, I check each variable for true `NA` values and then count how many times the level `"unknown"` appears in the categorical features, because `"unknown"` is used in this dataset to represent missing or unavailable information. After identifying these entries, I remove records with unknown education and impute unknown job values with the most frequent job category to create a cleaner dataset (`bank_clean`) for the next steps.[64] [65]

Code for task 1

Use a version **without comments** in the report if you want to keep your .R file comment-free:

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)

factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")
bank[factor_cols] <- lapply(bank[factor_cols], factor)

na_counts <- colSums(is.na(bank))
```

```
factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")
unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))

bank_clean <- bank
bank_clean <- subset(bank_clean, education != "unknown")

job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[^10_1]
bank_clean$job[bank_clean$job == "unknown"] <- job_mode

remaining_unknowns <- sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))
dim_bank_clean <- dim(bank_clean)
```

(If your teacher allows comments in the report code, you can keep your commented version there and leave this cleaner one in the `.R` file.)

Sample output of your code for task 1

You don't need to paste everything, just the key parts; for example:

```
na_counts
      age       job   marital education   default   balance   housing      loan   contact
        0         0         0         0         0         0         0         0         0
 campaign     pdays  previous  poutcome         y
        0         0         0         0         0

unknown_counts
      job   marital education   default   housing      loan   contact     month  poutcome
      288         0      1857         0         0         0     13020         0     36959

job_mode
[^10_1] "blue-collar"

remaining_unknowns
      job   marital education   default   housing      loan   contact     month  poutcome
        0         0         0         0         0         0     12343         0     35425

dim_bank_clean
[^10_1] 43354    17
```

(You can screenshot from R or copy–paste into Word; your choice.)

Code description of task 1

You can write this explanation in 5–7 sentences:

> First, I converted the main categorical variables (job, marital, education, default, housing, loan, contact, month, poutcome, and y) to factors so that they behave correctly as categorical attributes in later analyses. I then calculated `na_counts` using `colSums(is.na(bank))`, which showed that none of the 17 variables contain true `NA` values. Next, I used `unknown_counts` to count the number of `"unknown"` entries in each categorical variable and found that education, job, contact, and poutcome contain a substantial number of `"unknown"` values, indicating missing or unspecified information in those fields. To clean the data, I created a copy called `bank_clean`, removed all records with unknown education, and replaced unknown job values with the most frequent job category (`blue-collar`) as a simple mode-based imputation. After applying these steps, the cleaned dataset `bank_clean` has 43,354 records instead of 45,211, and there are no remaining unknown values in job or education, although contact and poutcome still contain some `"unknown"` levels that will be considered when interpreting later analyses. [65] [64]

You can adjust wording a bit to match your style, but keep the main points the same.

If you're ready, next I'll give you **Task 2:** Outlier detection and handling (with code and text exactly like this so you can drop it into the template).

⁂

yes sure proceed

Great. Let's make **Task 2: Outlier Detection and Handling** based on your cleaned data `bank_clean`.

We'll focus on two numeric variables that clearly have extreme values: **balance** and **duration** (you already saw very wide ranges in `summary(bank)`).[66]

1) Code for Task 2 (outlier detection & treatment)

Add this **after** your Task 1 code in `midterm_project.R`:

```
numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

summary(bank_clean[, numeric_cols])

balance_q1 <- quantile(bank_clean$balance, 0.25)
balance_q3 <- quantile(bank_clean$balance, 0.75)
balance_iqr <- balance_q3 - balance_q1
balance_lower <- balance_q1 - 1.5 * balance_iqr
balance_upper <- balance_q3 + 1.5 * balance_iqr

duration_q1 <- quantile(bank_clean$duration, 0.25)
duration_q3 <- quantile(bank_clean$duration, 0.75)
duration_iqr <- duration_q3 - duration_q1
duration_lower <- duration_q1 - 1.5 * duration_iqr
duration_upper <- duration_q3 + 1.5 * duration_iqr
```

```
balance_outliers_before <- sum(bank_clean$balance < balance_lower | bank_clean$balance >
duration_outliers_before <- sum(bank_clean$duration < duration_lower | bank_clean$durati

bank_clean2 <- bank_clean
bank_clean2$balance[bank_clean2$balance < balance_lower] <- balance_lower
bank_clean2$balance[bank_clean2$balance > balance_upper] <- balance_upper

bank_clean2$duration[bank_clean2$duration < duration_lower] <- duration_lower
bank_clean2$duration[bank_clean2$duration > duration_upper] <- duration_upper

balance_outliers_after <- sum(bank_clean2$balance < balance_lower | bank_clean2$balance >
duration_outliers_after <- sum(bank_clean2$duration < duration_lower | bank_clean2$durat

summary(bank_clean2[, c("balance", "duration")])
```

Run it and keep the console output; you'll paste parts into the report.

2) What output to show in the report

You don't need everything, just key parts such as:

- `summary(bank_clean[, numeric_cols])` (before capping).
- `balance_q1`, `balance_q3`, `balance_lower`, `balance_upper`.
- `duration_q1`, `duration_q3`, `duration_lower`, `duration_upper`.
- `balance_outliers_before`, `duration_outliers_before`.
- `balance_outliers_after`, `duration_outliers_after`.
- `summary(bank_clean2[, c("balance", "duration")])` (after capping).

For example (your numbers will be specific):

```
balance_q1
balance_q3
balance_lower
balance_upper

duration_q1
duration_q3
duration_lower
duration_upper

balance_outliers_before
duration_outliers_before

balance_outliers_after
duration_outliers_after

summary(bank_clean2[, c("balance", "duration")])
```

3) How to fill the template for Task 2

Title of the task 2

Detecting and Handling Outliers in Balance and Call Duration

Description of the task 2

> In this task, I detect and handle outliers in key numerical variables of the bank marketing dataset, focusing on account balance and call duration. These attributes show very wide ranges and extreme values, which can distort descriptive statistics and later analyses. I use the interquartile range (IQR) method to identify observations below the lower fence or above the upper fence and then cap those extreme values at the corresponding thresholds to reduce the influence of outliers while keeping all records in the dataset.[67] [66]

Code for task 2

Paste the code block from section 1 (without comments).

Sample output of your code for task 2

Paste selected console output, for example:

```
summary(bank_clean[, numeric_cols])

balance_q1
balance_q3
balance_lower
balance_upper

duration_q1
duration_q3
duration_lower
duration_upper

balance_outliers_before
duration_outliers_before

balance_outliers_after
duration_outliers_after

summary(bank_clean2[, c("balance", "duration")])
```

(Where the printed numbers show the fences and how many outliers were detected before and after.)

Code description of task 2

You can write something like this:

> First, I selected the main numerical variables (age, balance, duration, campaign, pdays, and previous) and examined their distributions using the `summary` function. The results showed very large ranges for balance and call duration, suggesting the presence of extreme values. I then applied the interquartile range (IQR) rule to both variables: for each one, I calculated the first quartile (Q1), third quartile (Q3), and IQR, and defined lower and upper fences as Q1 − 1.5 × IQR and Q3 + 1.5 × IQR. Using these thresholds, I counted how many observations fell outside the fences, which represented outliers in balance and duration. To mitigate their influence without dropping records, I created a new dataset `bank_clean2` and capped values below the lower fence and above the upper fence at the corresponding cutoff for each variable. After capping, the outlier counts for both balance and duration dropped to zero, and the summaries of these variables became less extreme while still preserving the overall shape of the data.[66] [67]

Next, if you confirm this runs fine, we'll:

- Fix `bank_clean2` as the main working dataset for the rest of the project.
- Create **Task 3**: type conversions (numeric ↔ categorical) plus a simple normalization task, both of which are required by your project description.

⁂

yes start and tell me how many tasks i need to do in total

You don't have a fixed required number of "tasks", but a good, clear project mapped to the teacher's bullet points would be about **9–10 tasks**:

1. Handling missing/"unknown" values (done).
2. Detecting and treating outliers (done).
3. Converting attributes between numeric and categorical.
4. Normalizing a continuous attribute.
5. Finding and removing duplicate rows.
6. Detecting and fixing invalid values / simple filtering.
7. Converting the imbalanced dataset into a balanced one.
8. Splitting data into training and testing sets.
9. Descriptive statistics by target classes (`y`).
10. Comparing means and variability across categories.[68] [69] [70]

Some of these can be grouped (e.g., 3+4 in one task, or 9+10 as separate tasks), but aiming for around 8–10 well-defined tasks is ideal.

Now we continue with **Task 3** (type conversions + normalization), using your cleaned data `bank_clean2`.

Task 3: Type conversion + normalization

We will:

- Create an **age group** variable (numeric → categorical).
- Convert a yes/no factor (e.g. `housing`) to numeric 0/1 (categorical → numeric).
- Normalize `duration` to a 0–1 scale.[69]

1) Code for Task 3

Add this code **after** Task 2 in your script:

```
bank_work <- bank_clean2

bank_work$age_group <- cut(
  bank_work$age,
  breaks = c(0, 30, 45, 60, Inf),
  labels = c("young", "middle", "mature", "senior"),
  right = TRUE
)

housing_num <- ifelse(bank_work$housing == "yes", 1, 0)
bank_work$housing_num <- housing_num

duration_min <- min(bank_work$duration)
duration_max <- max(bank_work$duration)
bank_work$duration_norm <- (bank_work$duration - duration_min) / (duration_max - duratic

str(bank_work[, c("age", "age_group", "housing", "housing_num", "duration", "duration_r
summary(bank_work[, c("age_group", "housing_num", "duration_norm")])
```

This gives you:

- `age_group`: factor with 4 levels.
- `housing_num`: numeric 0/1.
- `duration_norm`: normalized call duration between 0 and 1.[69]

2) What to show as sample output

From the console, use:

```
str(bank_work[, c("age", "age_group", "housing", "housing_num", "duration", "duration_r
summary(bank_work[, c("age_group", "housing_num", "duration_norm")])
```

Paste both outputs in the “Sample output” part for this task.

3) How to fill the template for Task 3

Title of the task 3
Converting Variable Types and Normalizing Call Duration

Description of the task 3

> In this task, I create new features by converting variables between numeric and categorical forms and by normalizing a continuous attribute. I group the age variable into four age categories to obtain an interpretable categorical feature, convert the housing loan status from a yes/no factor into a numeric 0–1 indicator, and scale the call duration variable to a 0–1 range so that it is comparable with other features and less affected by its large absolute values.[71] [69]

Code for task 3

Paste the Task 3 code block.

Sample output of your code for task 3

Paste the `str(...)` and `summary(...)` outputs showing:

- `age_group` as a factor.
- `housing_num` as numeric.
- `duration_norm` with values between 0 and 1.

Code description of task 3

You can write:

> I first created an `age_group` variable using the `cut` function, which bins the numeric age into four categories (young, middle, mature, senior) based on fixed age intervals. This transforms a continuous attribute into a categorical one that is easier to interpret when comparing client segments. Next, I converted the housing loan variable from a factor with levels “no” and “yes” into a numeric indicator `housing_num`, where 0 represents no housing loan and 1 represents having a housing loan. Finally, I normalized the call duration by subtracting its minimum value and dividing by the range, producing `duration_norm` values between 0 and 1. The structure and summary outputs confirm that the new variables have the expected types and ranges, and these transformed features will be used in later descriptive statistics and comparisons.[71] [69]

If this runs correctly, tell me, and next we’ll do:

- **Task 4:** Duplicates + invalid/filtered values.
- After that: balancing the dataset and train–test split, then the descriptive-statistics tasks.

⁂

proceed with task 4 to 6 i mean the next 3 tasks to do, one by one but in the same prompt

Nice, we'll build **Task 4, 5, and 6** on top of your existing objects (`bank_clean2` and `bank_work`). From now on we'll use a new object `bank_final` as the cleaned dataset we use for statistics.

Task 4 – Duplicates

Code for task 4

Add this after Task 3 in your script:

```
bank_nodup <- bank_work

dup_flags <- duplicated(bank_nodup)
dup_count <- sum(dup_flags)

dim_before_dup <- dim(bank_nodup)

bank_nodup <- bank_nodup[!dup_flags, ]

dim_after_dup <- dim(bank_nodup)

dup_count
dim_before_dup
dim_after_dup
```

Run it.

Sample output (what to paste)

From the Console, copy lines like:

```
dup_count
[^13_1] 0

dim_before_dup
[^13_1] 43354    20

dim_after_dup
[^13_1] 43354    20
```

(If `dup_count` is not 0 on your machine, just paste the real numbers.)

How to write Task 4 in the report

Title of the task 4
Detecting and Removing Duplicate Records

Description of the task 4

> In this task, I check whether the cleaned bank marketing dataset contains duplicate rows that could bias the analysis. Duplicate records may arise from repeated data collection or file merges and, if present, can overweight certain observations in descriptive statistics and later models. I use a built-in R function to identify duplicated rows and, if any exist, remove them to keep only unique client records.[84]

Code for task 4
Paste the code block above.

Sample output for task 4
Paste `dup_count`, `dim_before_dup`, and `dim_after_dup`.

Code description of task 4

> I created a copy of the working dataset called `bank_nodup` and used the `duplicated` function to generate a logical vector `dup_flags` indicating which rows are exact duplicates of previous rows. Summing this vector gives `dup_count`, the total number of duplicate records. I recorded the dataset dimensions before and after removing duplicated rows by subsetting `bank_nodup[!dup_flags, ]`. In my case, the number of duplicates was `dup_count`, and the dimensions before and after removal show that the dataset size did/not change, confirming that there were/no significant duplicate records after the earlier cleaning steps.[85]

> From this point onward, I use `bank_nodup` as the base dataset.

Task 5 – Invalid values and filtering

We will define and apply simple, reasonable validity rules based on the known description of this dataset:

- Age should be between 18 and 95 (dataset spec says ages are adult clients).[86] [84]
- Call duration should be strictly positive (duration 0 seconds is not informative and is treated here as invalid for analysis).[86]
- Campaign should be at least 1 contact (already true but we keep the rule).[84]

Code for task 5

Add this after Task 4:

```
bank_final <- bank_nodup

invalid_age <- sum(bank_final$age < 18 | bank_final$age > 95)
```

```
invalid_duration <- sum(bank_final$duration <= 0)
invalid_campaign <- sum(bank_final$campaign < 1)

invalid_age
invalid_duration
invalid_campaign

bank_final <- subset(
  bank_final,
  age >= 18 & age <= 95 &
    duration > 0 &
    campaign >= 1
)

dim_bank_final <- dim(bank_final)
summary(bank_final[, c("age", "duration", "campaign")])
```

Run it.

Sample output (what to paste)

From Console:

```
invalid_age
invalid_duration
invalid_campaign

dim_bank_final
[^13_1] xxxxx    20  # your actual numbers

summary(bank_final[, c("age", "duration", "campaign")])
```

How to write Task 5 in the report

Title of the task 5
Filtering Invalid Records Based on Age, Duration, and Campaign

Description of the task 5

> In this task, I detect and remove records with clearly invalid or uninformative values in key numerical variables of the bank marketing dataset. According to the data description, clients are adult individuals with ages roughly between 18 and 95, call duration is measured in seconds and should be positive, and the campaign variable counts the number of contacts in the current campaign. I treat ages outside the range 18–95, call durations equal to zero seconds, and campaign values below 1 as invalid for the purpose of this analysis and filter out such records to obtain a more reliable dataset. [84] [86]

Code for task 5
Paste the code block.

Sample output for task 5

Paste `invalid_age`, `invalid_duration`, `invalid_campaign`, `dim_bank_final`, and the `summary` output.

Code description of task 5

> I began with the de-duplicated dataset `bank_nodup` and copied it into `bank_final` for further cleaning. I then computed the counts of invalid values using logical conditions: ages below 18 or above 95, durations less than or equal to 0 seconds, and campaign counts less than 1. After checking how many such cases existed, I applied a `subset` filter requiring ages between 18 and 95, strictly positive call durations, and campaign values greater than or equal to 1. The resulting dimensions stored in `dim_bank_final` show that some records were removed, and the summaries of age, duration, and campaign confirm that the remaining data respect the defined validity rules. This cleaned dataset `bank_final` will be used in the balancing and splitting steps and for subsequent descriptive statistics.[86] [84]

Task 6 – Balancing classes and train–test split

Now we handle the class imbalance (y: many "no", fewer "yes") and create training and testing sets. We will:

- Randomly downsample the majority class "no" to match the number of "yes" observations (simple balancing).
- Create a 70% training / 30% testing split on the **balanced** dataset.[87] [84]

Code for task 6

Add this after Task 5:

```
table(bank_final$y)

set.seed(123)

yes_index <- which(bank_final$y == "yes")
no_index <- which(bank_final$y == "no")

n_yes <- length(yes_index)
n_no <- length(no_index)

n_yes
n_no

sampled_no_index <- sample(no_index, n_yes)

balanced_index <- c(yes_index, sampled_no_index)
bank_balanced <- bank_final[balanced_index, ]

bank_balanced$y <- droplevels(bank_balanced$y)

table(bank_balanced$y)
```

```
set.seed(123)

n_bal <- nrow(bank_balanced)
train_size <- floor(0.7 * n_bal)

train_indices <- sample(seq_len(n_bal), size = train_size)

train_data <- bank_balanced[train_indices, ]
test_data <- bank_balanced[-train_indices, ]

dim_train <- dim(train_data)
dim_test <- dim(test_data)

dim_train
dim_test
table(train_data$y)
table(test_data$y)
```

Run it.

Sample output (what to paste)

From Console:

```
table(bank_final$y)
n_yes
n_no
table(bank_balanced$y)

dim_train
dim_test
table(train_data$y)
table(test_data$y)
```

You should see that `bank_balanced` has equal counts for "yes" and "no", and training and test sets each keep that balance (or close to it because of random sampling).

How to write Task 6 in the report

Title of the task 6
Balancing the Target Classes and Splitting the Dataset into Training and Testing Sets

Description of the task 6

> In this task, I address the strong class imbalance in the target variable `y` and prepare the dataset for potential modeling by creating training and testing splits. In the original cleaned data, the majority of clients did not subscribe to a term deposit, so the number of "no" cases is much larger than the number of "yes" cases. I randomly downsample the majority class to match the number of "yes" observations, producing a balanced dataset, and then randomly split this balanced data into 70% training and 30% testing sets.[88] [87] [84]

Code for task 6
Paste the code block.

Sample output for task 6
Paste the output portions listed above.

Code description of task 6

> I first examined the class distribution of the cleaned dataset `bank_final` using `table(bank_final$y)`, which confirmed that "no" cases greatly outnumber "yes" cases. To balance the classes, I identified the indices of all "yes" and "no" observations, stored their counts in `n_yes` and `n_no`, and then used `sample` to randomly select exactly `n_yes` indices from the majority "no" group. Combining all "yes" indices with the sampled "no" indices created `bank_balanced`, a dataset with equal numbers of positive and negative outcomes, and I removed unused factor levels from `y` using `droplevels`. Next, I set a random seed for reproducibility and drew a 70% random sample of row indices from `bank_balanced` to define `train_data`, with the remaining 30% forming `test_data`. The printed dimensions and class frequency tables for the training and testing sets show that both subsets are of the expected size and retain the balanced distribution between "yes" and "no" clients.[88] [84]

With Tasks 1–6, you've covered:

- Missing/unknown values
- Outliers
- Type conversions and normalization
- Duplicates
- Invalid values/filtering
- Balancing and splitting

Next, we'll focus on the **descriptive statistics and comparisons** required in the assignment:

- Task 7: Descriptive statistics of numerical variables by target class `y`.
- Task 8: Compare mean of a numeric variable across two categories.
- Task 9: Compare variability (IQR, sd, variance, range) across multiple categories.

⁂

This is what i have done till now, proceed with the next 4 steps

Nice, you're in a good place. We'll do the **next 4 tasks** as:

- Task 7: Descriptive statistics by target class `y`.
- Task 8: Compare mean of a numeric variable across two categories.
- Task 9: Compare variability of another numeric variable across categories.

- Task 10: Overall descriptive summary of the cleaned dataset (ties everything together).

We'll use `bank_final` (cleaned, unbalanced dataset) so the stats reflect the real data, not the downsampled one.[89] [90]

Task 7 – Descriptive statistics by target classes (`y`)

Code for task 7

Add this after Task 6 in your script:

```
numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

stats_mean_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = mean
)

stats_median_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = median
)

stats_sd_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = sd
)

stats_mean_by_y
stats_median_by_y
stats_sd_by_y
```

Sample output to paste

From the Console, copy the printed tables, for example:

```
stats_mean_by_y
stats_median_by_y
stats_sd_by_y
```

(You don't have to paste all three if it's too long; at least mean and sd per class.)

How to fill the report

Title of the task 7
Descriptive Statistics of Numerical Variables by Term-Deposit Outcome

Description of the task 7

> In this task, I compute descriptive statistics for the main numerical variables of the bank marketing dataset separately for clients who subscribed to a term deposit (`y = "yes"`) and those who did not (`y = "no"`). By comparing group-wise means, medians, and standard deviations for age, account balance, call duration, campaign contacts, days since last contact, and number of previous contacts, I can understand how these characteristics differ between positive and negative outcomes.[90] [89]

Code for task 7
Paste the code block.

Sample output of your code for task 7
Paste the printed `stats_mean_by_y`, `stats_median_by_y`, and `stats_sd_by_y`.

Code description of task 7

> I defined a vector of key numerical attributes (`age`, `balance`, `duration`, `campaign`, `pdays`, and `previous`) and used the `aggregate` function with `y` as the grouping factor to calculate mean, median, and standard deviation for each variable within the "yes" and "no" classes. The resulting tables show, for example, whether successful clients tend to have longer call durations, higher or lower balances, or different campaign contact patterns compared to clients who did not subscribe. Comparing these statistics helps identify which features are most strongly associated with the subscription outcome and provides a quantitative summary of differences between the two groups.[91] [89] [90]

Task 8 – Compare average of one numeric variable across two categories

We'll compare the **average call duration** between clients with and without a housing loan (categorical `housing`, levels `"no"` / `"yes"`).[89]

Code for task 8

Add this:

```
mean_duration_by_housing <- aggregate(
  duration ~ housing,
  data = bank_final,
  FUN = mean
)

mean_duration_by_housing
```

```
duration_ttest <- t.test(duration ~ housing, data = bank_final)
duration_ttest
```

Sample output to paste

From the Console:

```
mean_duration_by_housing
duration_ttest
```

How to fill the report

Title of the task 8
Comparing Average Call Duration for Clients With and Without Housing Loans

Description of the task 8

> In this task, I compare the average duration of marketing calls between two categories of the housing loan variable: clients with a housing loan (`housing = "yes"`) and clients without a housing loan (`housing = "no"`). This addresses the requirement to compare the mean of a numerical variable across two distinct categories of a categorical attribute in the dataset.[90] [89]

Code for task 8
Paste the code block.

Sample output of your code for task 8
Paste the table printed by `mean_duration_by_housing` and the output of `duration_ttest`.

Code description of task 8

> I used `aggregate(duration ~ housing, ...)` to calculate the mean call duration separately for clients who do and do not have a housing loan, obtaining two group means that can be directly compared. To formally test whether the difference in average duration is statistically significant, I applied a two-sample t-test using `t.test(duration ~ housing, data = bank_final)`. The test output reports the estimated mean difference, confidence interval, and p-value; together with the group means, this shows whether the bank typically spends more time on calls to clients with or without housing loans and whether the observed difference is likely to be due to chance. This satisfies the project requirement to compare average values of a numerical variable across two categories of a categorical variable.[89] [90]

Task 9 – Compare variability across categories

Now we compare the **variability of account balance** across the age groups you created (`age_group`: young, middle, mature, senior).[92] [89]

We will compute IQR, standard deviation, variance, and range for `balance` within each age group.

Code for task 9

Add this:

```
balance_IQR_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = IQR
)

balance_sd_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = sd
)

balance_var_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = var
)

balance_range_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = function(x) max(x) - min(x)
)

balance_IQR_by_agegroup
balance_sd_by_agegroup
balance_var_by_agegroup
balance_range_by_agegroup
```

Sample output to paste

From the Console:

```
balance_IQR_by_agegroup
balance_sd_by_agegroup
balance_var_by_agegroup
balance_range_by_agegroup
```

How to fill the report

Title of the task 9
Comparing Variability of Account Balance Across Age Groups

Description of the task 9

> In this task, I examine and compare the variability of the account balance variable across the different age groups created earlier (young, middle, mature, and senior). This addresses the requirement to analyze measures of spread—such as interquartile range, standard deviation, variance, and range—for a numerical attribute across multiple categories of a categorical variable.[92] [89]

Code for task 9
Paste the code block.

Sample output of your code for task 9
Paste the four tables printed for IQR, standard deviation, variance, and range.

Code description of task 9

> Using the `aggregate` function with `age_group` as the grouping factor, I computed the interquartile range, standard deviation, variance, and range of account balance for each age category. The IQR reflects the spread of the middle 50% of balances, the standard deviation and variance capture overall dispersion around the mean, and the range summarizes the distance between minimum and maximum balances in each group. Comparing these values across the four age groups reveals which segment of clients exhibits the most volatile or concentrated balances—for example, whether seniors have more widely dispersed balances than younger clients. This helps the bank understand which age segments show greater financial variability and may require different marketing or risk strategies.[93] [92] [89]

Task 10 – Overall descriptive summary of the cleaned dataset

This final task is a short "wrap-up" descriptive summary of the main cleaned dataset `bank_final`. It's not explicitly required but makes the report look complete and connects all earlier cleaning steps.

Code for task 10

Add:

```
overall_summary_numeric <- summary(bank_final[, c("age", "balance", "duration",
                                                  "campaign", "pdays", "previous")])
overall_summary_numeric

overall_summary_categorical <- summary(bank_final[, c("job", "marital", "education",
                                                      "default", "housing", "loan",
```

```
                                                    "contact", "month", "poutcome", "y")
overall_summary_categorical
```

(If the second summary is too wide, you can print subsets, e.g., 4–5 variables at a time.)

Sample output to paste

From the Console:

```
overall_summary_numeric
overall_summary_categorical
```

(You can paste only the most important parts into the report.)

How to fill the report

Title of the task 10
Overall Descriptive Summary of the Cleaned Dataset

Description of the task 10

> In this task, I present an overall descriptive summary of the final cleaned dataset before performing any advanced modeling. After handling missing and unknown values, outliers, invalid records, duplicates, and class imbalance, it is important to verify the central tendencies and distributions of key variables and confirm that the data are consistent with expectations from the original dataset description.[90] [89]

Code for task 10
Paste the code block.

Sample output of your code for task 10
Paste the main parts of `overall_summary_numeric` and `overall_summary_categorical`.

Code description of task 10

> I used the `summary` function on the numerical variables to obtain their minimum, quartiles, median, mean, and maximum values in the final cleaned dataset `bank_final`. These summaries confirm that age values are within a realistic adult range, balances and call durations are positive, and campaign and previous contact counts are nonnegative after filtering. I also summarized the categorical attributes, which shows the frequency distribution of job types, marital status, education levels, loan statuses, contact methods, months, and campaign outcomes. Together, these summaries provide a compact but comprehensive view of the cleaned dataset and confirm that the data are ready for interpretation and potential modeling steps.[93] [89]

If you run all this and it works, your coding side is essentially complete for the project.
Next, we can:

- Check your report file (task titles and descriptions).

- Make sure you have the “Project Code” section filled with your full script.
- Verify that the ZIP (R file + PDF + sample CSV) matches your teacher’s submission instructions.

⁂

This is my current status, all done? everything that was asked have i done all to get the fulls. I am giving you the question again.

"Introduction to Data Science [Section: C]
Midterm Project - Spring 25-26

Project Description
Apply the appropriate data preparation steps to your chosen dataset and compute the relevant descriptive statistics. For this project, each group must work with its own selected dataset and modify it according to the project requirements. You may choose any labelled dataset containing both categorical and numerical attributes (classes will be categorical) that fits the task except iris, titanic, and US Arrest datasets. The project must be completed in a four-person group. A spreadsheet is created in MS Teams to provide all group and project information. The link to the file is given in the Teams group. While giving entry to the spreadsheet. ensure that no other group has already selected your dataset for the project.
Instructions
• The submission deadline for the project is April 5, 2026, 11:59 pm.
• At the beginning of the report, write a short note about the dataset. For each implemented code segment in the R program, provide the code and its output along with their description in the report. In the description part, only write the content (do not write unnecessary content) that is sufficient to understand the code and its output.
• A file related to the report format is provided in MS Teams. Use that file to write the report.
• Comments are not allowed in the R program.
• I will announce the project VIVA schedule in MS Teams. Please bring your implemented program during the VIVA session.
• An assignment is created in MS Teams named “Project - Midterm.” Make a ZIP file using your project-related files (R file, Project Report in PDF, and Sample Dataset file) and submit it under that assignment by the deadline. One student from each group will submit the project.
• The following topics can be focused on thinking about the project. Note that the project is not limited to these topics which are mentioned to get an idea about how to proceed with the project.
◦ If there are any missing values in the dataset, we should apply all applicable methods from the available options to handle the missing values.
◦ Detect outliers in the data set and use the appropriate approach to handle those values.
◦ We can convert attributes from numeric to categorical or categorical to numeric.
◦ We can apply the normalization method for any continuous attribute.
◦ We can find and remove duplicate rows.
◦ We can apply some filtering methods to filter the data.

◦ Detect invalid data in the data set and use the appropriate approach to handle those values.
◦ We can convert the imbalanced data set into a balanced data set.
◦ Split the dataset for Training and Testing.
◦ Calculate descriptive statistics and interpret the results for the numerical variables according to the target classes of your chosen dataset.
◦ Compare the average values of a selected numerical variable across two distinct categories of an appropriate categorical variable from your dataset.
◦ Examine and compare the variability (e.g., IQR,standard deviation, variance, and range) of another numerical variable across the different categories or levels of a chosen categorical variable within your dataset."

You are **very close**, but I would not say "fully done" yet. You have completed the core analysis steps, but there are still a few things to verify before submission.[95]

What you have already completed

- Dataset selected and loaded.
- Categorical variables converted to factors.
- Missing / `"unknown"` values handled.
- Outliers detected and capped.
- Numeric/categorical conversion and normalization done.
- Duplicates checked and removed if needed.
- Invalid data filtered.
- Class balancing and train/test split done.
- Descriptive statistics by target and comparison tasks done.[95]

What still needs to be checked

Report completeness

Your report must include, for **each task**:

- Title of the task.
- Short description.
- Code.
- Sample output.
- Code description.[95]

So the main thing left is making sure the **Word report is fully filled** and not missing any task block.

Full code in the report

The template asks for the **entire code of the project** at the end.[95]

So you need to paste your **final complete R script** in the "Project Code" section.

R script must be clean

The instruction says **comments are not allowed in the R program**.[95]

That means your final `.R` file should ideally have no `#` comments before submission.

Dataset sample file

You also need to submit a **sample dataset file** in the ZIP.[95]

PDF export

You need to export the report to PDF before zipping.[95]

Important thing to fix

Your current workflow includes **both** `bank_clean2`, `bank_work`, `bank_nodup`, `bank_final`, **and** `bank_balanced`, which is fine, but the report should clearly say which object is used for which task.

For example:

- `bank` = original data
- `bank_clean` = after missing/unknown handling
- `bank_clean2` = after outlier handling
- `bank_work` = after type conversion and normalization
- `bank_nodup` = after duplicate check
- `bank_final` = after invalid filtering
- `bank_balanced` = after balancing
- `train_data` / `test_data` = after split

That makes your report and viva easier.

My honest answer

Yes, the main project tasks are done.

No, you are not yet at the final submission stage until you do these:

- fill the report fully,
- paste the full code,
- export PDF,
- prepare sample dataset,
- ZIP the three files,
- and double-check the output of each task in the report.

What I recommend next

I can help you with one of these now:

1. **Make a final checklist for submission.**
2. **Give you the final clean R code with no comments**, ready to paste into the script.
3. **Help you write the missing report text for each task block.**
4. **Help you prepare the ZIP file contents exactly.**

If you want, I can do the **final checklist + final clean R script** next.

⁂

yes full R code doing the same thing without comments submission ready give me this and then tell me what to do next

Below is a **submission-ready full R script without comments.** It keeps the same workflow we built step by step.

Full R code

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)

factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")
bank[factor_cols] <- lapply(bank[factor_cols], factor)

na_counts <- colSums(is.na(bank))

unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))

bank_clean <- bank
```

```
bank_clean <- subset(bank_clean, education != "unknown")

job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[^16_1]
bank_clean$job[bank_clean$job == "unknown"] <- job_mode

remaining_unknowns <- sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))

balance_q1 <- quantile(bank_clean$balance, 0.25)
balance_q3 <- quantile(bank_clean$balance, 0.75)
balance_iqr <- balance_q3 - balance_q1
balance_lower <- balance_q1 - 1.5 * balance_iqr
balance_upper <- balance_q3 + 1.5 * balance_iqr

duration_q1 <- quantile(bank_clean$duration, 0.25)
duration_q3 <- quantile(bank_clean$duration, 0.75)
duration_iqr <- duration_q3 - duration_q1
duration_lower <- duration_q1 - 1.5 * duration_iqr
duration_upper <- duration_q3 + 1.5 * duration_iqr

balance_outliers_before <- sum(bank_clean$balance < balance_lower | bank_clean$balance >
duration_outliers_before <- sum(bank_clean$duration < duration_lower | bank_clean$durati

bank_clean2 <- bank_clean
bank_clean2$balance[bank_clean2$balance < balance_lower] <- balance_lower
bank_clean2$balance[bank_clean2$balance > balance_upper] <- balance_upper
bank_clean2$duration[bank_clean2$duration < duration_lower] <- duration_lower
bank_clean2$duration[bank_clean2$duration > duration_upper] <- duration_upper

balance_outliers_after <- sum(bank_clean2$balance < balance_lower | bank_clean2$balance >
duration_outliers_after <- sum(bank_clean2$duration < duration_lower | bank_clean2$durat

bank_work <- bank_clean2

bank_work$age_group <- cut(
  bank_work$age,
  breaks = c(0, 30, 45, 60, Inf),
  labels = c("young", "middle", "mature", "senior"),
  right = TRUE
)

bank_work$housing_num <- ifelse(bank_work$housing == "yes", 1, 0)

duration_min <- min(bank_work$duration)
duration_max <- max(bank_work$duration)
bank_work$duration_norm <- (bank_work$duration - duration_min) / (duration_max - duratic

bank_nodup <- bank_work

dup_flags <- duplicated(bank_nodup)
dup_count <- sum(dup_flags)
dim_before_dup <- dim(bank_nodup)

bank_nodup <- bank_nodup[!dup_flags, ]
dim_after_dup <- dim(bank_nodup)

bank_final <- bank_nodup
```

```
invalid_age <- sum(bank_final$age < 18 | bank_final$age > 95)
invalid_duration <- sum(bank_final$duration <= 0)
invalid_campaign <- sum(bank_final$campaign < 1)

bank_final <- subset(
  bank_final,
  age >= 18 & age <= 95 &
    duration > 0 &
    campaign >= 1
)

dim_bank_final <- dim(bank_final)

table(bank_final$y)

set.seed(123)

yes_index <- which(bank_final$y == "yes")
no_index <- which(bank_final$y == "no")

n_yes <- length(yes_index)
n_no <- length(no_index)

sampled_no_index <- sample(no_index, n_yes)

balanced_index <- c(yes_index, sampled_no_index)
bank_balanced <- bank_final[balanced_index, ]

bank_balanced$y <- droplevels(bank_balanced$y)

set.seed(123)

n_bal <- nrow(bank_balanced)
train_size <- floor(0.7 * n_bal)

train_indices <- sample(seq_len(n_bal), size = train_size)

train_data <- bank_balanced[train_indices, ]
test_data <- bank_balanced[-train_indices, ]

numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

stats_mean_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = mean
)

stats_median_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = median
)

stats_sd_by_y <- aggregate(
```

```
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = sd
)

mean_duration_by_housing <- aggregate(
  duration ~ housing,
  data = bank_final,
  FUN = mean
)

duration_ttest <- t.test(duration ~ housing, data = bank_final)

balance_IQR_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = IQR
)

balance_sd_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = sd
)

balance_var_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = var
)

balance_range_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = function(x) max(x) - min(x)
)

overall_summary_numeric <- summary(bank_final[, c("age", "balance", "duration",
                                                   "campaign", "pdays", "previous")])

overall_summary_categorical <- summary(bank_final[, c("job", "marital", "education",
                                                       "default", "housing", "loan",
                                                       "contact", "month", "poutcome", "y")
```

What to do next

1. Put this code into your `.R` file

- Open your R script.
- Replace the old code with this full version.
- Save it.

2. Run it once

Make sure there are no errors in the Console.

3. Export your report as PDF

In the Word template:

- Fill in all task sections.
- Paste the relevant code blocks, outputs, and short descriptions.
- Add the full code at the end in the **Project Code** section.
- Save as PDF.

4. Make your sample dataset file

Use a small sample from your cleaned dataset, for example:

- first 100 rows of `bank_final`
- save as CSV

If you want, I can give you the exact R code to create that sample CSV too.

5. Zip the files

Your ZIP should contain:

- `midterm_project.R`
- `project_report.pdf`
- sample dataset CSV

6. Submit to MS Teams

Submit under **Project - Midterm** before the deadline.

If you want, my next message can be:

- the **exact code to create the sample dataset file**, or
- a **final submission checklist**, or
- a **clean task-by-task report text** you can paste into the Word template.

⁂

your code is only 167 line but my code was 256 lines

"# Working directory is already set via Session -> Set Working Directory

Now load the full bank marketing dataset

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)

dim(bank)
names(bank)
str(bank)
summary(bank)
```

Convert character variables to factors (categorical)

```
factor_cols <- c("job", "marital", "education", "default",
"housing", "loan", "contact", "month",
"poutcome", "y")

bank[factor_cols] <- lapply(bank[factor_cols], factor)

str(bank)

table(bank$y)
prop.table(table(bank$y))
```

Check true NA values

```
na_counts <- colSums(is.na(bank))
na_counts
```

Check 'unknown' levels in categorical variables

```
factor_cols <- c("job", "marital", "education", "default",
"housing", "loan", "contact", "month",
"poutcome", "y")

unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))
unknown_counts

bank_clean <- bank
```

Remove rows with unknown education

```
bank_clean <- subset(bank_clean, education != "unknown")
```

Impute unknown job with most frequent job

```
job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[1]
job_mode

bank_clean$job[bank_clean$job == "unknown"] <- job_mode
```

Check remaining unknowns

sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))

New dimension after cleaning

dim(bank_clean)

numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

summary(bank_clean[, numeric_cols])

balance_q1 <- quantile(bank_clean$balance, 0.25)$balance_q3 <- -quantile(bank_clean$balance, 0.75)
balance_iqr <- balance_q3 - balance_q1
balance_lower <- balance_q1 - 1.5 * balance_iqr
balance_upper <- balance_q3 + 1.5 * balance_iqr

duration_q1 <- quantile(bank_clean$duration, 0.25)$duration_q3 <- -quantile(bank_clean$duration, 0.75)
duration_iqr <- duration_q3 - duration_q1
duration_lower <- duration_q1 - 1.5 * duration_iqr
duration_upper <- duration_q3 + 1.5 * duration_iqr

balance_outliers_before <- sum(bank_clean$balance < $balance_lower|bank_clean$balance > balance_upper)
duration_outliers_before <- sum(bank_clean$duration < $duration_lower|bank_clean$duration > duration_upper)

bank_clean2 <- bank_clean
bank_clean2$balance[$bank_clean2$balance < balance_lower] <- balance_lower
bank_clean2$balance[$bank_clean2$balance > balance_upper] <- balance_upper

bank_clean2$duration[$bank_clean2$duration < duration_lower] <- duration_lower
bank_clean2$duration[$bank_clean2$duration > duration_upper] <- duration_upper

balance_outliers_after <- sum(bank_clean2$balance < $balance_lower|bank_clean2$balance > balance_upper)
duration_outliers_after <- sum(bank_clean2$duration < $duration_lower|bank_clean2$duration > duration_upper)

summary(bank_clean2[, c("balance", "duration")])

bank_work <- bank_clean2

bank_work$age_group <- -cut($bank_work$age,
breaks = c(0, 30, 45, 60, Inf),
labels = c("young", "middle", "mature", "senior"),
right = TRUE
)

housing_num <- ifelse(bank_work$housing == "yes", 1, 0)$bank_work$housing_num <- housing_num

duration_min <- min(bank_work$duration)$duration_max <- max(bank_work$duration)
bank_work$duration_norm <- (bank_work$duration - duration_min) / (duration_max - duration_min)

str(bank_work[, c("age", "age_group", "housing", "housing_num", "duration", "duration_norm")])
summary(bank_work[, c("age_group", "housing_num", "duration_norm")])

bank_nodup <- bank_work

dup_flags <- duplicated(bank_nodup)
dup_count <- sum(dup_flags)

dim_before_dup <- dim(bank_nodup)

bank_nodup <- bank_nodup[!dup_flags,]

dim_after_dup <- dim(bank_nodup)

dup_count
dim_before_dup
dim_after_dup

bank_final <- bank_nodup

invalid_age <- sum(bank_final$age < 18 | bank_final$age > 95)
invalid_duration <- sum(bank_final$duration <= 0)$invalid_campaign <- sum(bank_final$campaign < 1)

invalid_age
invalid_duration
invalid_campaign

bank_final <- subset(
bank_final,
age >= 18 & age <= 95 &
duration > 0 &
campaign >= 1
)

dim_bank_final <- dim(bank_final)
summary(bank_final[, c("age", "duration", "campaign")])

table(bank_final$y)

set.seed(123)

yes_index <- which(bank_final$y == "yes")$no_index <- which(bank_final$y == "no")

```
n_yes <- length(yes_index)
n_no <- length(no_index)

n_yes
n_no

sampled_no_index <- sample(no_index, n_yes)

balanced_index <- c(yes_index, sampled_no_index)
bank_balanced <- bank_final[balanced_index, ]

bank_balanced$y <- droplevels(bank_balanced$y)

table(bank_balanced$y)

set.seed(123)

n_bal <- nrow(bank_balanced)
train_size <- floor(0.7 * n_bal)

train_indices <- sample(seq_len(n_bal), size = train_size)

train_data <- bank_balanced[train_indices, ]
test_data <- bank_balanced[-train_indices, ]

dim_train <- dim(train_data)
dim_test <- dim(test_data)

dim_train
dim_test
table(train_data$y)table(test_data$y)

numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

stats_mean_by_y <- aggregate(
bank_final[, numeric_cols],
by = list(y = bank_final$y),
FUN = mean
)

stats_median_by_y <- aggregate(
bank_final[, numeric_cols],
by = list(y = bank_final$y),
FUN = median
)

stats_sd_by_y <- aggregate(
bank_final[, numeric_cols],
by = list(y = bank_final$y),
FUN = sd
)
```

```
stats_mean_by_y
stats_median_by_y
stats_sd_by_y

mean_duration_by_housing <- aggregate(
duration ~ housing,
data = bank_final,
FUN = mean
)

mean_duration_by_housing

duration_ttest <- t.test(duration ~ housing, data = bank_final)
duration_ttest

balance_IQR_by_agegroup <- aggregate(
balance ~ age_group,
data = bank_final,
FUN = IQR
)

balance_sd_by_agegroup <- aggregate(
balance ~ age_group,
data = bank_final,
FUN = sd
)

balance_var_by_agegroup <- aggregate(
balance ~ age_group,
data = bank_final,
FUN = var
)

balance_range_by_agegroup <- aggregate(
balance ~ age_group,
data = bank_final,
FUN = function(x) max(x) - min(x)
)

balance_IQR_by_agegroup
balance_sd_by_agegroup
balance_var_by_agegroup
balance_range_by_agegroup

overall_summary_numeric <- summary(bank_final[, c("age", "balance", "duration",
"campaign", "pdays", "previous")])
overall_summary_numeric
```

```
overall_summary_categorical <- summary(bank_final[, c("job", "marital", "education",
"default", "housing", "loan",
"contact", "month", "poutcome", "y")])
overall_summary_categorical"
```

Your full script is fine — the reason it looks longer is that it includes every step explicitly, and that's okay. The exact line count does **not** matter for submission; what matters is that the script is clean, runs without errors, and includes all required tasks.[96]

What you should do now

1. Keep your longer script

Use your current version, because it is more complete and more closely matches the project requirements. Your script already covers:

- loading and type conversion,
- missing/unknown handling,
- outlier treatment,
- transformation and normalization,
- duplicates,
- invalid filtering,
- balancing and split,
- descriptive statistics and comparisons.[97]

2. Remove all comments from the final `.R` file

The instructor said comments are not allowed in the R program, so before final submission, delete every line starting with `#`.[97]

3. Keep the actual code blocks

Do **not** remove the code that creates objects like `na_counts`, `unknown_counts`, `balance_q1`, `stats_mean_by_y`, etc. Those are part of the task output and help the viva.

Best final R script

Your current script is already basically the final version. The only thing I would suggest is:

- Keep it as one script.
- Remove comments.
- Make sure it runs from top to bottom with no errors.

So instead of replacing it with a shorter version, use **your own working script** as the final one.

What to do next

Step 1

Open your R script and remove every comment line starting with `#`.

Step 2

Run the entire script from top to bottom to confirm there are no errors.

Step 3

In the Word template:

- Fill every task block.
- Paste the code snippets and outputs.
- Put the full final code in the **Project Code** section.

Step 4

Export the report as PDF.

Step 5

Create a sample CSV from the cleaned data.

If you want, I can now give you:

1. a **cleaned final version of your exact script with all comments removed**, or
2. the **R code to save a sample dataset CSV**, or
3. a **final submission checklist**.

⁂

1. IDS-Report-Template-with-Cover-Page-Mid-term-Spring-25-26.docx
2. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
3. https://fairlearn.org/main/api_reference/generated/fairlearn.datasets.fetch_bank_marketing.html
4. https://www.kaggle.com/datasets/adityamhaske/bank-marketing-dataset/code
5. https://www.kaggle.com/datasets/adebayoadebanjo/uci-bank-marketing-dataset
6. Project-Information-Midterm-IDS-C-Spring-25-26.xlsx
7. https://github.com/JosePadillaMtnz/BreastCancer_KaggleDataset
8. https://henriquelaureano.github.io/courses/ml-kaust/project_report.pdf
9. https://www.kaggle.com/c/bank-marketing-uci/overview
10. https://askai.glarity.app/search/What-are-some-Kaggle-datasets-available-for-breast-cancer-imaging-classification

11. https://github.com/Reza-Marzban/Bank-Marketing-Data-Set-Classification/blob/master/README.md
12. https://www.kaggle.com/datasets/bletchley/bank-marketing
13. https://analyst-2.ai/analysis/kaggle-breast-cancer-diagnostic-dataset-bcd-63e2/50e77951/?iid=012-966&v=presentation
14. https://github.com/alexkataev/Case-Study-UCI-Bank-Marketing-Dataset
15. https://www.geeksforgeeks.org/machine-learning/ml-kaggle-breast-cancer-wisconsin-diagnosis-using-knn/
16. https://www.scribd.com/document/649141003/Classification-Bank-Marketing-Dataset-Jupyter-Notebook
17. https://github.com/kaimzhao/kaggle-breast-cancer-uci-diagnosis-data-set
18. https://ubc-library-rc.github.io/IntroR/content/RStudio.html
19. https://wiingy.com/learn/r-studio/difference-between-r-programming-and-rstudio/
20. https://spout.uits.iu.edu/CRAN/bin/windows/base/rw-FAQ.R-4.5.3patched.html
21. https://cran.r-project.org/bin/windows/base/
22. https://www.reddit.com/r/Rlanguage/comments/1c3cv0n/r_studio_doesnt_have_new_project_tab/
23. https://forum.posit.co/t/the-advantages-of-rstudio/120416
24. https://www.youtube.com/watch?v=Kef6bqeDidg
25. https://www.youtube.com/watch?v=BvqGwsmsIgk
26. https://www.youtube.com/watch?v=IJc2J-qewiU
27. https://stat.ethz.ch/R-manual/R-patched/RHOME/doc/NEWS.pdf
28. https://support.posit.co/hc/en-us/articles/200526207-Using-RStudio-Projects
29. https://becker.wustl.edu/news/how-to-use-rstudio-projects-to-structure-r-working-directories/
30. https://www.youtube.com/watch?v=H9EBlFDGG4k
31. https://docs.posit.co/ide/user/ide/guide/code/projects.html
32. https://www.reddit.com/r/RStudio/comments/1egbzzf/r_vs_r_studio/
33. https://spout.uits.iu.edu/CRAN/bin/windows/base/rw-FAQ.R-4.5.3patched.html
34. https://wiingy.com/learn/r-studio/difference-between-r-programming-and-rstudio/
35. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
36. https://fairlearn.org/main/api_reference/generated/fairlearn.datasets.fetch_bank_marketing.html
37. image.jpg
38. https://ubc-library-rc.github.io/IntroR/content/RStudio.html
39. https://library.virginia.edu/data/research-software/r-and-r-studio
40. https://posit.co/download/rstudio-desktop/
41. https://helpdesk.tcsedsystem.edu/hc/en-us/articles/13485648405783-What-is-R-Studio-Desktop-and-how-do-I-install-it
42. https://dailies.rstudio.com/links/
43. https://www.youtube.com/watch?v=9SzKJH93t5o
44. https://pesktop.com/ru/windows/posit-rstudio-desktop

45. https://www.youtube.com/watch?v=F1Y8Rmee_Fk
46. https://rstudio-desktop.en.softonic.com
47. https://ro.scribd.com/document/475953289/Installing-R-and-R-studio-docx
48. https://posit.co/downloads/
49. https://www.youtube.com/watch?v=iHrJTzYVFNw
50. https://teacherscollege.screenstepslive.com/a/1108074-install-r-and-rstudio-for-windows
51. https://www.youtube.com/watch?v=H9EBlFDGG4k
52. https://rstudio-education.github.io/hopr/starting.html
53. https://www.maxplaining.com/blog/rstudio-installation
54. https://support.posit.co/hc/en-us/articles/200526207-Using-RStudio-Projects
55. https://docs.posit.co/ide/user/ide/guide/code/projects.html
56. https://ubc-library-rc.github.io/IntroR/content/RStudio.html
57. https://spout.uits.iu.edu/CRAN/bin/windows/base/rw-FAQ.R-4.5.3patched.html
58. https://support.posit.co/hc/en-us/articles/200526207-Using-RStudio-Projects
59. https://docs.posit.co/ide/user/ide/guide/code/projects.html
60. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
61. https://fairlearn.org/main/api_reference/generated/fairlearn.datasets.fetch_bank_marketing.html
62. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
63. https://fairlearn.org/main/api_reference/generated/fairlearn.datasets.fetch_bank_marketing.html
64. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
65. https://fairlearn.org/main/api_reference/generated/fairlearn.datasets.fetch_bank_marketing.html
66. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
67. https://fairlearn.org/main/api_reference/generated/fairlearn.datasets.fetch_bank_marketing.html
68. https://archive.ics.uci.edu/dataset/222/bank+marketing
69. https://www.kaggle.com/datasets/hariharanpavan/bank-marketing-dataset-analysis-classification
70. IDS-Report-Template-with-Cover-Page-Mid-term-Spring-25-26.docx
71. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
72. https://help.qlik.com/talend/en-US/connectors-guide/Cloud/inserting-filtered-data-into-azure-cosmos-db-table?l=zh-CN
73. https://github.com/Rawanmalharbi/UCI-Bank-Marketing
74. https://www.kaggle.com/discussions/general/354614
75. https://docs.profiling.ydata.ai/4.14/examples/bank_marketing_data/uci_bank_marketing_report.html
76. https://www.kaggle.com/datasets/berkayalan/bank-marketing-data-set
77. https://www.kaggle.com/competitions/aicvs-bank/data
78. https://www.kaggle.com/datasets/ashnaimtiaz/bank-marketing-uci-dataset
79. https://studylib.net/doc/28037805/bank-data-set-variables-description

80. https://byui-cse.github.io/cse450-course/module-02/bank-dictionary.txt
81. https://github.com/alexkataev/Case-Study-UCI-Bank-Marketing-Dataset
82. https://search.r-project.org/CRAN/refmans/fairml/html/bank.html
83. https://www.studocu.vn/vn/document/truong-dai-hoc-kinh-te-thanh-pho-ho-chi-minh/khoa-hoc-du-lieu/bank-marketing-dataset-overview-uci-ml-repo/145391339
84. https://archive.ics.uci.edu/dataset/222/bank+marketing
85. https://search.r-project.org/CRAN/refmans/fairml/html/bank.html
86. https://studylib.net/doc/28037805/bank-data-set-variables-description
87. IDS-Report-Template-with-Cover-Page-Mid-term-Spring-25-26.docx
88. https://www.kaggle.com/datasets/hariharanpavan/bank-marketing-dataset-analysis-classification
89. https://archive.ics.uci.edu/dataset/222/bank+marketing
90. https://www.kaggle.com/datasets/hariharanpavan/bank-marketing-dataset-analysis-classification
91. https://search.r-project.org/CRAN/refmans/fairml/html/bank.html
92. https://studylib.net/doc/28037805/bank-data-set-variables-description
93. https://docs.profiling.ydata.ai/4.14/examples/bank_marketing_data/uci_bank_marketing_report.html
94. IDS-Report-Template-with-Cover-Page-Mid-term-Spring-25-26.docx
95. IDS-Report-Template-with-Cover-Page-Mid-term-Spring-25-26.docx
96. https://rmarkdown.rstudio.com/articles_report_from_r_script.html
97. IDS-Report-Template-with-Cover-Page-Mid-term-Spring-25-26.docx
98. https://stackoverflow.com/questions/12673697/code-appears-as-comments-in-r
99. https://www.kaggle.com/datasets/volodymyrgavrysh/bank-marketing-campaigns-dataset/metadata
100. https://forum.posit.co/t/rstudio-ide-fails-to-comment-uncomment-certain-selected-lines-in-a-r-markdown-file/111357
101. https://www.programiz.com/r/comments
102. https://docs.1010data.com/Tutorials/MachineLearningExamples/BankMarketingDataSet_2.html
103. https://github.com/rstudio/rstudio/issues/3891
104. https://www.scribd.com/document/473582742/Payment-Gateway-Report
105. https://www.w3schools.com/r/r_comments.asp
106. https://www.geeksforgeeks.org/r-language/comments-in-r/
107. https://www.scaler.com/topics/comments-in-r/
108. https://blog.r-hub.io/2023/01/26/code-comments-self-explaining-code/
109. https://sqlpad.io/tutorial/comments-r-beginners-guide/
110. https://www.kaggle.com/datasets/kevalm/bank-marketingdataset
111. https://stackoverflow.com/questions/41896493/script-comments-in-rstudio-not-displaying-in-the-console